

LAPORAN PROGRESS PENGEMBANGAN SISTEM

AuditChain Gateway — Blockchain-based Audit Trail Middleware

Nama Proyek: AuditChain Gateway

Jenis Sistem: Middleware Audit Trail berbasis Blockchain (Hyperledger Fabric)

Periode Laporan: [isi tanggal]

Disusun oleh: [isi nama]

1. RINGKASAN PROYEK

AuditChain Gateway adalah sistem middleware yang berfungsi sebagai API Gateway untuk menerima, memproses, dan mengunci log audit dari berbagai sistem klien (SIMRS/rumah sakit, sistem geospasial, dsb.) secara *immutable* ke dalam jaringan Hyperledger Fabric Blockchain. Sistem ini menggunakan arsitektur multi-tenant (SaaS), mendukung ingestion data melalui Change Data Capture (CDC) berbasis Kafka/Debezium per klien, serta menerapkan mekanisme verifikasi integritas data berlapis (multi-layer verification).

[PLACEHOLDER GAMBAR 1: Diagram arsitektur umum sistem AuditChain Gateway]

2. TUJUAN PENGEMBANGAN

1. Menjamin integritas data pada sistem klien tanpa mengubah kode aplikasi existing (zero code intrusion).
 2. Menyediakan audit trail yang tidak dapat dimanipulasi (tamper-evident) menggunakan kombinasi hashing kriptografis (SHA3-256), Merkle Tree, dan blockchain anchoring.
 3. Memberikan dashboard monitoring dan verifikasi bagi auditor dan admin.
 4. Mendukung berbagai jenis basis data klien (PostgreSQL, MySQL, Oracle, MongoDB, SQL Server).
-

3. ARSITEKTUR SISTEM

Sistem terdiri dari beberapa komponen utama:

- **Ingestion Layer** — menerima data perubahan (INSERT/UPDATE/DELETE) dari database klien melalui Kafka CDC (Debezium) maupun HTTP API.

- **Hasher Engine** — menghitung hash SHA3-256 dari setiap log yang masuk.
- **Aggregator Engine** — mengelompokkan log menjadi batch dan membangun Merkle Tree.
- **Fabric Anchoring** — mengirimkan Merkle Root ke jaringan Hyperledger Fabric.
- **Verification Engine (4 Layer)** — memverifikasi integritas data secara berlapis (DB Existence → Re-Hash → Agent Source → Blockchain Consensus).
- **Dashboard Web (React)** — antarmuka bagi admin dan auditor untuk memonitor dan memverifikasi data.

[PLACEHOLDER GAMBAR 2: Diagram alur data (data flow) dari database klien hingga blockchain]

[PLACEHOLDER GAMBAR 3: Diagram lifecycle status log (RECEIVED → HASHED → AGGREGATED → ANCHORED)]

4. PROGRESS YANG TELAH DICAPAI

4.1 Modul Autentikasi & Multi-tenant

- Implementasi autentikasi tiga skema: API Key (klien → gateway), JWT Bearer (dashboard), dan Verify Token (gateway → agent klien).
- Pendaftaran klien/tenant baru beserta pembuatan API Key otomatis.

4.2 Modul Ingestion

- Ingestion data melalui Kafka CDC per klien dengan reconciliation otomatis setiap 15 detik.
- Endpoint HTTP ingestion (</api/logs>) untuk jalur alternatif berbasis Redis Queue.

4.3 Modul Kriptografi & Blockchain

- Hashing SHA3-256 terhadap 8 field kanonik log audit.
- Pembangunan Merkle Tree dengan dukungan proof chain lengkap dari leaf ke root (termasuk kasus leaf ganjil/self-pairing).
- Anchoring Merkle Root ke jaringan Hyperledger Fabric.

4.4 Modul Verifikasi

- Verifikasi 4 lapis: eksistensi database, re-hashing lokal, verifikasi ke Agent klien, dan pencocokan Merkle Proof terhadap ledger Fabric.
- Dashboard menampilkan status integritas granular: **valid**, **tampered**, **pending**, **unreachable**.

4.5 Dashboard & Admin Panel

- Statistik ringkasan (total log, pending, anchored).
- Riwayat transaksi dengan pagination, filter, dan pencarian.
- Manajemen klien, konfigurasi Kafka, dan konfigurasi Agent Lapis 3.
- Panel monitoring CDC user klien.

[PLACEHOLDER GAMBAR 4: Screenshot dashboard utama (Audit Logs / Overview)]

[PLACEHOLDER GAMBAR 5: Screenshot Admin Panel — Client Registry]

5. UPDATE TERBARU

5.1 Penambahan Kolom Pemetaan Actor pada Pendaftaran Klien

Pada update ini, dilakukan penyempurnaan terhadap proses **registrasi klien baru** dengan menambahkan kolom-kolom konfigurasi pemetaan field *actor* (pelaku perubahan data) yang lebih granular berdasarkan jenis operasi (INSERT, UPDATE, DELETE). Sebelumnya, sistem hanya menggunakan satu field actor umum (dengan opsi fallback). Melalui update ini, admin dapat mengisi field mapping khusus untuk setiap jenis aksi, sehingga pencatatan pelaku pada log audit menjadi lebih akurat sesuai struktur tabel database masing-masing klien.

Kolom baru yang ditambahkan pada form pendaftaran klien:

Kolom	Fungsi
Insert Actor	Field yang digunakan untuk mengambil nilai actor saat terjadi operasi INSERT pada tabel klien
Update Actor	Field yang digunakan untuk mengambil nilai actor saat terjadi operasi UPDATE pada tabel klien
Delete Actor	Field yang digunakan untuk mengambil nilai actor saat terjadi operasi DELETE pada tabel klien
Placeholder Actor	Nilai contoh/petunjuk pengisian bagi admin, misalnya <code>user_id</code> , untuk memudahkan pengisian field mapping yang sesuai dengan skema tabel klien

Tujuan penambahan fitur ini:

1. Mengakomodasi kasus di mana kolom pelaku perubahan berbeda-beda tergantung jenis operasi (misalnya tabel klien mencatat `created_by` untuk INSERT, `updated_by` untuk UPDATE, dan `deleted_by` untuk DELETE).

2. Meningkatkan akurasi data `actor` yang tercatat pada `audit_logs`, khususnya untuk klien dengan struktur tabel yang tidak seragam.
3. Memberikan panduan pengisian yang lebih jelas bagi admin melalui *placeholder* pada form pendaftaran, sehingga mengurangi kesalahan konfigurasi field mapping saat onboarding klien baru.

[PLACEHOLDER GAMBAR 6: Screenshot form "Register Client" dengan kolom Insert Actor, Update Actor, Delete Actor, dan Placeholder Actor]

[PLACEHOLDER GAMBAR 7: Screenshot hasil log audit yang menunjukkan nilai actor sesuai jenis aksi setelah update]

6. KENDALA & CATATAN TEKNIS

- Beberapa endpoint ingestion HTTP (`internal/modules/ingestion`) masih tersedia namun belum terhubung penuh ke router utama.
- Perlu penyesuaian lanjutan pada consumer Kafka/CDC agar dapat membaca field mapping actor per-aksi yang baru ditambahkan, memastikan konsistensi antara data yang tersimpan di `clients` dengan proses normalisasi log pada `kafkaconsumer` dan `normalizer`.
- Validasi lanjutan diperlukan agar fallback tetap berjalan jika field mapping per-aksi kosong (menggunakan actor field umum sebagai default).

7. RENCANA SELANJUTNYA

1. Integrasi penuh mapping actor per-aksi ke pipeline Kafka Consumer dan Ingestion Handler.
2. Pengujian end-to-end terhadap klien dengan struktur tabel actor yang berbeda-beda.
3. Penyempurnaan dokumentasi API dan panduan onboarding klien baru.
4. Review keamanan pra-produksi (CORS whitelist, validasi JWT secret, dsb).

[PLACEHOLDER GAMBAR 8: Diagram rencana pengembangan / roadmap]

8. LAMPIRAN

[PLACEHOLDER GAMBAR 9: Lampiran tambahan sesuai kebutuhan]